Aim:Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.

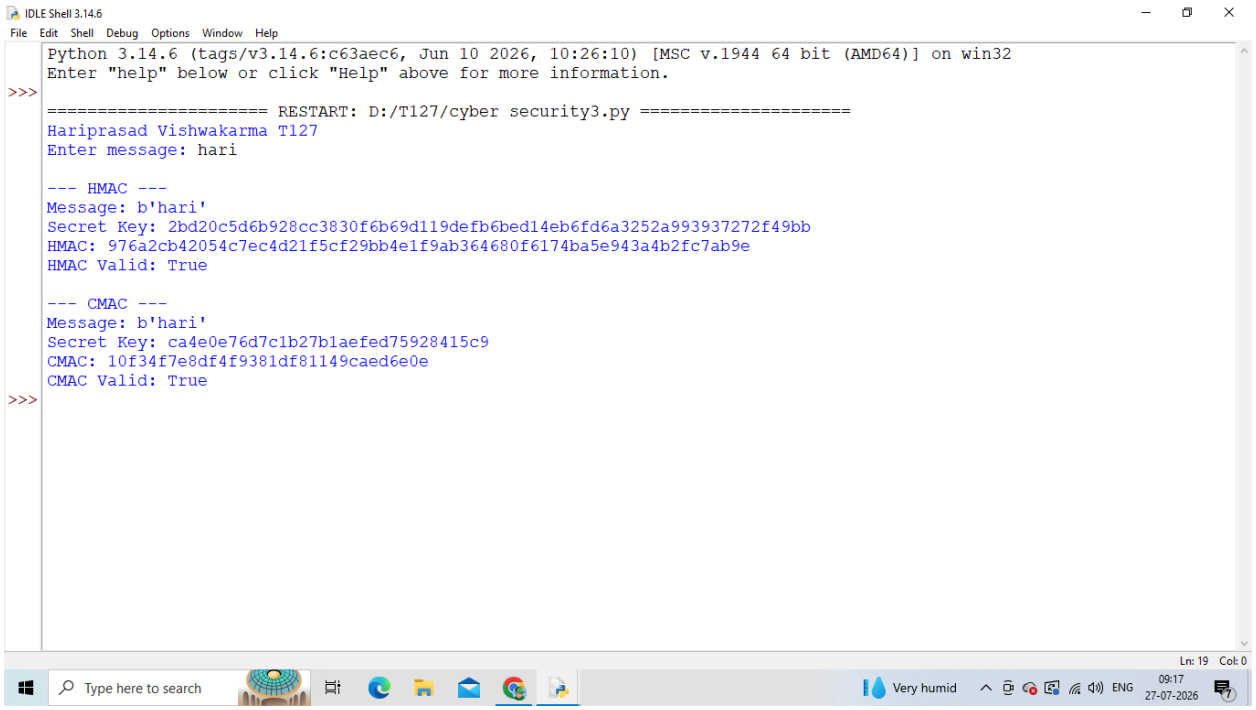
Code:

```
import hmac
import hashlib
import secrets
from cryptography.hazmat.primitives import cmac
from cryptography.hazmat.primitives.ciphers import algorithms

# --------- HMAC (hash-based MAC) ---------
print("Hariprasad Vishwakarma T127")

def generate_hmac(message: bytes):
    key = secrets.token_bytes(32)
    tag = hmac.new(key, message, hashlib.sha256).digest()
    print("Message:", message)
    print("Secret Key:", key.hex())
    print("HMAC:", tag.hex())
    return key, tag

def verify_hmac(key: bytes, message: bytes, tag: bytes) -> bool:
    valid = hmac.compare_digest(hmac.new(key, message, hashlib.sha256).digest(), tag)
    print("HMAC Valid:", valid)
    return valid


# --------- CMAC (AES block-cipher-based MAC) ---------

def generate_cmac(message: bytes):
    key = secrets.token_bytes(16)  # AES-128 key
    c = cmac.CMAC(algorithms.AES(key))
    c.update(message)
    tag = c.finalize()
    print("Message:", message)
    print("Secret Key:", key.hex())
    print("CMAC:", tag.hex())
    return key, tag

def verify_cmac(key: bytes, message: bytes, tag: bytes) -> bool:
    c = cmac.CMAC(algorithms.AES(key))
    c.update(message)
    try:
        c.verify(tag)
        print("CMAC Valid: True")
        return True
```

Hariprasad Vishwakarma
T127
TYCS

```
    except Exception:
        print("CMAC Valid: False")
        return False


# --------- Run: one message, generate then verify ---------

text = input("Enter message: ")
message = text.encode()

print("\n--- HMAC ---")
key1, tag1 = generate_hmac(message)
verify_hmac(key1, message, tag1)

print("\n--- CMAC ---")
key2, tag2 = generate_cmac(message)
verify_cmac(key2, message, tag2)
```

Output:



Code 4:

```
# --------- HMAC (hash-based MAC) ---------

def generate_hmac(message: bytes):
    key = secrets.token_bytes(32)
    tag = hmac.new(key, message, hashlib.sha256).digest()
```

Hariprasad Vishwakarma
T127
TYCS

```
    return key, tag

def verify_hmac(key: bytes, message: bytes, tag: bytes) -> bool:
    return hmac.compare_digest(hmac.new(key, message, hashlib.sha256).digest(), tag)


# ---------- CMAC (AES block-cipher-based MAC) ----------

def generate_cmac(message: bytes):
    key = secrets.token_bytes(16)  # AES-128 key
    c = cmac.CMAC(algorithms.AES(key))
    c.update(message)
    tag = c.finalize()
    return key, tag

def verify_cmac(key: bytes, message: bytes, tag: bytes) -> bool:
    c = cmac.CMAC(algorithms.AES(key))
    c.update(message)
    try:
        c.verify(tag)
        return True
    except Exception:
        return False


# ---------- shared state (holds last generated keys/tags) ----------

state = {"message": None, "hmac_key": None, "hmac_tag": None,
         "cmac_key": None, "cmac_tag": None}


# ---------- GUI actions ----------

def do_generate():
    text = entry.get().strip()
    if not text:
        messagebox.showwarning("Input needed", "Please enter a message.")
        return
    message = text.encode()

    hkey, htag = generate_hmac(message)
    ckey, ctag = generate_cmac(message)

    state["message"] = message
```

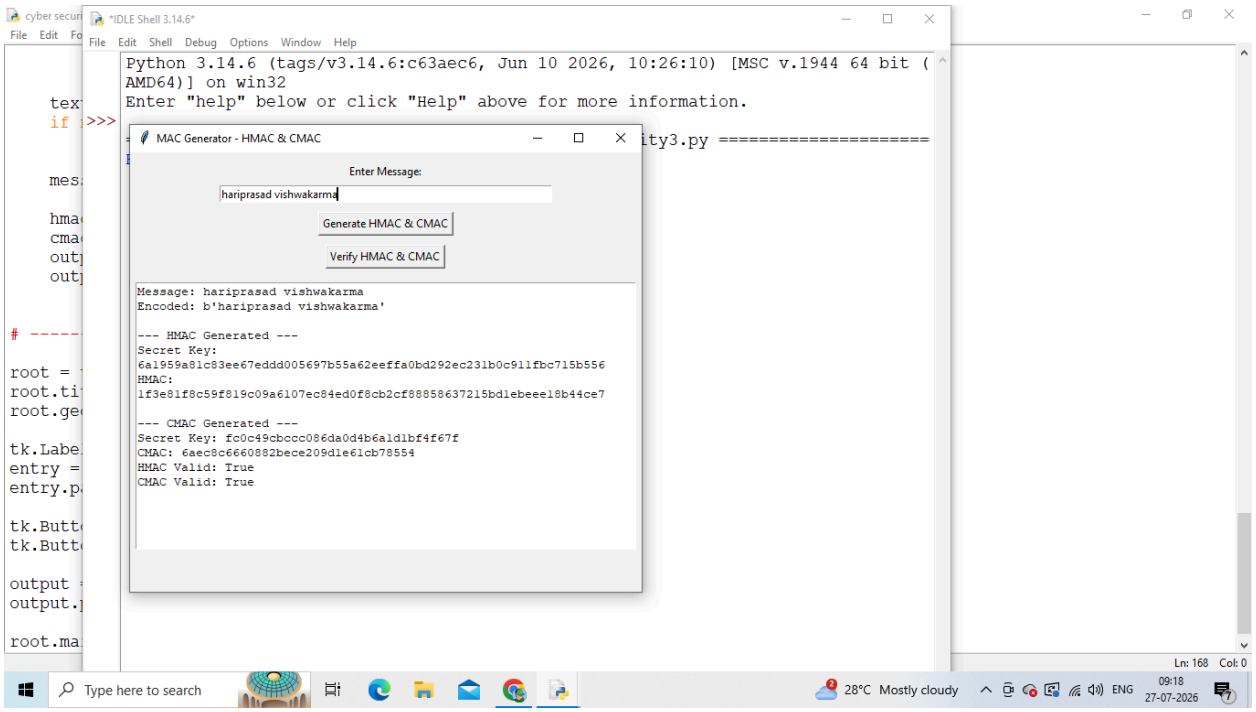
Hariprasad Vishwakarma
T127
TYCS

```
    state["hmac_key"], state["hmac_tag"] = hkey, htag
    state["cmac_key"], state["cmac_tag"] = ckey, ctag

    output.delete("1.0", tk.END)
    output.insert(tk.END, f"Message: {text}\n")
    output.insert(tk.END, f"Encoded: {message}\n\n")

    output.insert(tk.END, "--- HMAC Generated ---\n")
    output.insert(tk.END, f"Secret Key: {hkey.hex()}\n")
    output.insert(tk.END, f"HMAC: {htag.hex()}\n\n")

    output.insert(tk.END, "--- CMAC Generated ---\n")
    output.insert(tk.END, f"Secret Key: {ckey.hex()}\n")
    output.insert(tk.END, f"CMAC: {ctag.hex()}\n")


def do_verify():
    if state["hmac_key"] is None:
        messagebox.showwarning("Nothing to verify", "Generate the MACs first.")
        return

    text = entry.get().strip()
    if not text:
        messagebox.showwarning("Input needed", "Please enter a message to verify.")
        return
    message = text.encode()  # re-read current box content, not the stored one

    hmac_valid = verify_hmac(state["hmac_key"], message, state["hmac_tag"])
    cmac_valid = verify_cmac(state["cmac_key"], message, state["cmac_tag"])
    output.insert(tk.END, f"HMAC Valid: {hmac_valid}\n")
    output.insert(tk.END, f"CMAC Valid: {cmac_valid}\n")


# --------- GUI layout ---------

root = tk.Tk()
root.title("MAC Generator - HMAC & CMAC")
root.geometry("560x480")

tk.Label(root, text="Enter Message:").pack(pady=(10, 0))
entry = tk.Entry(root, width=60)
entry.pack(pady=5)

tk.Button(root, text="Generate HMAC & CMAC", command=do_generate).pack(pady=5)
```

Hariprasad Vishwakarma
T127
TYCS

```
tk.Button(root, text="Verify HMAC & CMAC", command=do_verify).pack(pady=5)

output = tk.Text(root, width=68, height=18, wrap="word")
output.pack(pady=10)

root.mainloop()
```

Output:



Hariprasad Vishwakarma
T127
TYCS